Carrom Club — Technical Documentation

A cross-platform multiplayer carrom game: web, mobile, an authoritative game server, and an operator console. This document records what was built, the technology and infrastructure it runs on, and how each part of it works.

Last verified: **122 unit tests** and **388 end-to-end checks** across six suites, all passing against a live stack, covering **162 HTTP routes** and every socket event — plus a concurrent-load run holding a 30 ms median shot latency.

---

1. What this is

Carrom Club is a real-time, skill-based multiplayer game. Two or four players sit at a virtual carrom board, flick a striker, and pocket their men. The physics run identically on every device, the server decides what actually happened, and the money on the table is virtual.

Three non-negotiable rules shape the whole system:

1. **No gambling.** There is no real-money betting, no wagering, no cash deposits, no withdrawals and no cash prizes. Coins and gems are virtual items with no cash value, and there is no code path anywhere that converts them back to money.
2. **The client is never trusted.** It never decides a shot's outcome, a match's winner, or that a purchase succeeded. It sends intent; the server decides.
3. **Nothing bought affects a match.** Every purchasable is currency, a cosmetic, a crate or a convenience. A player who spends nothing can beat a player who spends everything.

---

2. Technology

Languages and runtime

LAYER	TECHNOLOGY	WHY
Everything	**TypeScript 5.6**, strict mode	One language across server, web, mobile and shared logic — the physics engine is literally the same file on all three
Runtime	**Node.js 22 LTS**	Native `fetch`, stable ESM
Package management	**npm workspaces**	No extra tooling; the shared packages are plain local dependencies

Backend

CONCERN	TECHNOLOGY
HTTP API	**Express 4**
Real-time	**Socket.IO 4** (WebSocket, long-poll fallback)
Database	**PostgreSQL 18**
DB driver	**pg** (connection pool, no ORM)
Cache / presence / queues	**Redis 7** via **ioredis**
Validation	**zod** — every request body and query string
Auth	**jsonwebtoken** (JWT) + **bcryptjs**
Logging	**pino** (structured JSON)
Security headers	**helmet**
Tests	**vitest**

No ORM is used deliberately. Every query is visible SQL, which matters for a system whose correctness claims are about ledgers and race conditions.

Web client

CONCERN	TECHNOLOGY
Framework	**Next.js 14** (App Router)
UI	**React 18**
Styling	**Tailwind CSS 3**, palette driven by CSS custom properties
Rendering	**HTML Canvas 2D**
Transport	**socket.io-client**

Mobile client

CONCERN	TECHNOLOGY
Framework	**Expo SDK 52 / React Native 0.76**
Navigation	**React Navigation 6** (bottom tabs)
Rendering	**react-native-svg**
Web preview	**react-native-web** — the mobile app also runs in a browser
Storage	**AsyncStorage**
Auth	**expo-auth-session** for Google, Facebook SDK for Facebook

Operator console

Next.js 14 + Tailwind, sharing the same design language. It holds no privileged secret of its own — staff sign in with normal accounts and the API refuses every admin route unless the account carries the `moderator` or `admin` role.

3. Repository layout

```
carrom/
├── apps/
│   ├── api/          Express + Socket.IO server        (51 files, ~13,800 lines)
│   ├── web/          Next.js player client             (33 files,  ~8,800 lines)
│   ├── mobile/       Expo / React Native client        (14 files,  ~3,300 lines)
│   └── admin/        Next.js operator console          (14 files,  ~3,200 lines)
├── packages/
│   ├── types/        Shared domain types
│   ├── config/       Tiers, timers, XP curve, ranks, rewards
│   ├── physics/      Deterministic rigid-body simulation
│   ├── game-engine/  Carrom rules on top of the physics
│   ├── content/      191 cosmetics, achievements, store catalogue, 143 bot names
│   └── ui/           One renderer, two painters (Canvas and SVG)
└── infrastructure/
    ├── database/     11 forward-only SQL migrations, 70 tables
    ├── docker/       Postgres + Redis for local development
    └── deployment/   Container and environment configuration
```

The two ideas that make this layout work

One physics engine, three platforms. `packages/physics` and `packages/game-engine` are pure TypeScript with no platform dependencies. The server imports them to be authoritative; the web and mobile clients import the same files to predict locally. Because it is the same code with a fixed timestep, a client's prediction and the server's authority agree frame for frame.

One renderer, two backends. `packages/ui` defines a `Painter` interface implemented by `CanvasPainter` (web) and `SvgPainter` (mobile). Every board, striker and coin is drawn once against that interface, so 20 boards and 191 cosmetics have a single drawing implementation rather than two that drift.

---

4. The game

Physics

A deterministic rigid-body simulation at a **fixed 1/120s timestep**.

- Circle-to-circle elastic collisions with per-body mass
- Surface friction and cushion restitution
- Pocket detection by proximity to the four corner wells
- Bodies come to rest below a velocity threshold

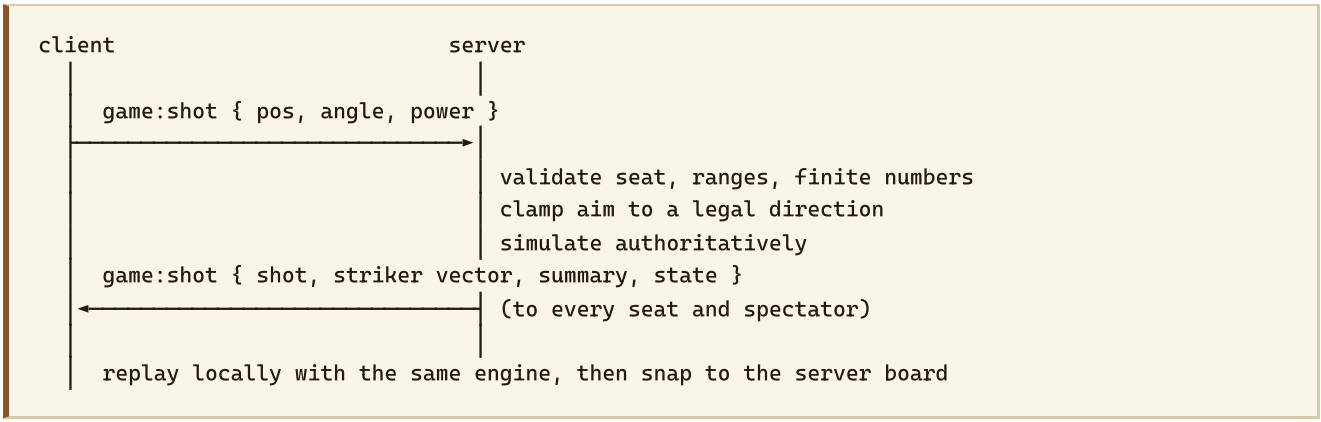
Determinism is the load-bearing property. It is what allows the client to replay a shot locally and arrive at the same board the server did, which is why the game feels instant while remaining server-authoritative.

Rules

Implemented in `packages/game-engine/src/rules.ts`:

- Turn order and colour assignment by seat
- **Queen must be covered** — pocket the red queen and you must pocket one of your own men on the same or the next turn, or she returns to the centre
- Fouls: pocketing the striker, no contact, touching nothing of your own
- Penalty pieces returned to the board, tracked as a debt when there is nothing to return
- Repeat turn on a legal pocket
- **150-turn cap** with a points decision, so no board can run forever

The shot protocol



The client sends only three numbers. The server broadcasts the shot **plus the exact launch vector it used**, so every client replays the identical simulation and no one can desync by rounding differently.

Anti-cheat

- Every shot is validated server-side before it is simulated
- Rate limits on shots, chat and every mutating HTTP route
- Timing analysis and win-rate review feed a `cheat_flags` queue
- **The system never bans on its own.** Flags are evidence for a human moderator, who decides.

---

5. Features

Accounts and sessions

- Email + password, Facebook, Google, and guest play
- Guests can upgrade to a full account and keep everything
- JWT access tokens (short-lived, in memory) + refresh tokens (rotating, single-use — a reused refresh token is rejected and the family revoked)
- **OAuth secrets never reach the frontend.** Clients send a provider token; the server exchanges and verifies it.

Matchmaking and play

- Quick, Classic, Ranked, Private room (join by code), Friend match, Team (2v2) and solo Practice
- Six coin tiers from Beginner (200) to Grandmaster (1,000,000)
- Skill-banded matchmaking with a tolerance that widens the longer you wait
- Reconnection with full state resync; AFK detection with turn timeouts
- Spectator mode with per-match viewer limits and reactions

Economy

Two currencies, both **virtual and non-redeemable**:

- **Coins** — earned by playing, spent on entries and cosmetics
- **Gems** — premium, bought or earned, spent on cosmetics and conveniences

Both use an **append-only ledger**. Every movement writes a ledger row in the same transaction as the balance change, and the balance update carries its own guard (`balance + delta >= 0`) so a debit can never take an account negative even under concurrent requests.

`GET /admin/economy/audit` and `GET /admin/monetization/gem-audit` verify that every balance equals the sum of its ledger. **Both currently report zero mismatches.**

Progression

- XP and levels with 8 titles (Beginner → Legend)
- Trophies and skill rating; ranked divisions Bronze → Legend
- Seasons with themes, soft rating reset and end-of-season rewards
- 17 achievements, 13 daily/weekly missions
- 7-day login calendar with streaks and correct missed-day handling
- 8 collections covering **191 collectibles**, with completion milestones

Cosmetics

191 items, none of which touch the physics engine: 50 strikers, 50 coin sets, 20 boards, 24 avatars, 16 frames, 12 banners, 11 badges, 8 victory animations.

They are stored as **drawing recipes, not images** — colours, a pattern name and an animation record. Nothing downloads, everything scales to any screen, and no cosmetic field is ever read by the simulation.

Crates

Won from matches, with unlock timers by rarity. Five rarities (Common → Legendary) with published drop weights. Duplicates convert to coins and crafting fragments.

Social

- Friends, friend requests, blocking
- Follow players without needing approval
- Presence: Online / In game / In matchmaking / Away / Offline

- Preset quick chat and emoji reactions — **no free text anywhere**, so there is nothing to moderate
- Server-enforced emote limits: 1.5s cooldown, 8 per minute, 30s timeout on breach, plus a recipient-side mute
- Fair play score, leaderboards (global / country / friends), tournaments with knockout brackets

Computer opponents

A player should never stare at an empty queue, and should never be stuck without someone to practise against. Both are covered by the same bot.

How it plays. The bot picks a shot by *playing it*: it clones the board, runs the real engine on each candidate, and scores what actually happened. It is bound by exactly the rules a human is — it cannot see through pieces, cannot place the striker illegally, and cannot escape a foul it caused. There is no separate "bot physics" to drift out of step with the game.

The search is aimed rather than exhaustive: candidates are generated from striker positions that actually line up with a target man, so a small time budget still finds sensible shots. It yields between batches, so a bot thinking never stalls another match on the same node.

Three difficulties, produced by *degrading* the result rather than by giving the bot less information — so a beginner-level bot misses the way a beginner does:

	SEARCH	AIM ERROR	BLUNDER RATE	THINKING TIME
Easy	3 positions, ±1 fan	0.075 rad	35%	1.6–3.4 s
Medium	5 positions, ±2 fan	0.032 rad	15%	1.3–2.8 s
Hard	7 positions, ±3 fan	0.012 rad	4%	0.9–2.2 s

Measured over 14 boards, **hard beats easy 11–3**, with a 15% foul rate and about 25 shots a board — so the ladder means something.

Where they appear.

- **Practice, by choice.** Pick Easy, Medium or Hard on the Play screen, on web or mobile. Unstaked, so it starts instantly and pays only token XP.
- **Matchmaking fallback.** After **12 seconds** with nobody to pair with, a bot is seated instead. Its difficulty is chosen to suit the player's level and trophies. This match *is* staked.

Bots are real accounts. Each has a user row, a profile and a coin balance, so it pays its entry fee and collects its winnings through the same append-only ledger everyone else uses. A bot match therefore **moves** coins rather than minting them, and the economy audit needs no special case. Top-ups are a normal ledger movement with reason `bot_float`, so the coins bots bring to the table are visible and countable — over time they lose more than they win, which is the intended sink.

Names. Drawn from a pool of **143 human first names** across South Asia, East and Southeast Asia, Africa, Europe, the Middle East, Latin America and the Anglophone world, with a surname initial about a third of the time. The roster is generated from a **fixed seed**, so it is the same roster on every boot — without that,

seeding would create a fresh set of accounts on every restart. Thirty-six bots are seeded, twelve per difficulty, each with a level and trophy count drawn from a band matching how it actually plays.

Bots are always disclosed. Every seat carries `isBot` and `botSkill`, and both clients show a `BOT` label with the difficulty. They are excluded from leaderboards, from search, and from presence.

Coin gifts between friends

One friend sending another virtual coins, once a day.

A gift **moves** coins; it never creates them. The sender is debited and the recipient credited inside one transaction, so both balances still equal the sum of their ledgers afterwards.

The thing this is designed against is alt-account farming — making throwaway accounts and funnelling their starting coins into a main. Four things stand in the way:

- **One gift per sender per day**, enforced by a unique index on `(from_user, sent_on)` rather than a read-then-write, so two devices tapping at once cannot both succeed.
- **A capped amount** (500–25,000 coins), and the sender must keep at least 1,000.
- **Full accounts only.** A guest cannot gift, which is what makes throwaway accounts useless for it.
- **Friends of at least 24 hours**, so an account cannot be created and drained the same hour.

Bots cannot be gifted to, and every transfer is recorded for moderation.

Tournaments

Knockout brackets, seeded by trophies, with byes for the top seeds when the entry count is not a power of two. Entry is a virtual coin fee paid once at registration; the pool is the prize.

The bracket runs itself. Once a pairing has both players and both are connected, the server seats them automatically — a tournament that waits on somebody clicking "ready" is a tournament that stalls on one absent player. When the board ends, the result is reported back, the winner advances, and the final pays the pool to the champion.

Two details worth stating, because both are easy to get wrong:

- **A bracket board is unstaked.** The entry was taken at registration, so charging again at the table would take a second fee from both players.
- **A pairing is claimed before it is seated.** Only the update that moves it out of `ready` returns a row, so two nodes cannot seat the same match twice; a failed seating hands the claim back rather than stranding the bracket.

Moderation

Player reports → moderator review queue → decision → ban, with an audit log of every staff action. Bans are scoped (account, ranked, or chat) and can expire.

---

6. Monetization

Store

Twelve products across bundles, coin packs, gem packs, the season pass, a membership subscription and permanent ad removal. Prices live in the database so a sale needs no deploy, held in **minor currency units** (paise) as integers — never floats.

The purchase flow

```
1. POST /store/checkout   → creates a `pending` row, returns the platform SKU.
                            Takes no money. Grants nothing.
2. [platform payment sheet runs — Apple, Google or Stripe]
3. POST /store/confirm    → server verifies the receipt WITH THE PROVIDER,
                            compares product and price against its own row,
                            then grants inside one transaction.
```

What the server refuses:

- A receipt the provider does not confirm
- A receipt for a different product than the order
- A receipt whose amount does not match our price
- A receipt already used (unique index on `(provider, provider_txn)` — a replay from a second device collides rather than granting twice)
- Any purchase at all when no payment provider is configured — a misconfigured production deploy refuses to sell rather than giving things away

Sandbox payments exist for development and are **hard-disabled in production**.

Carrom Pass

A 50-tier season pass with a free track that pays on **every** tier and a premium track unlocked by purchase. Buying it unlocks the premium track **retroactively**, so a player who buys at tier 30 immediately receives tiers 1–30. The final premium tier is a legendary crate.

Claims are idempotent by primary key: a double tap inserts once.

Subscriptions

Buy, renew, cancel, resume, expire. Three rules shape it:

1. **A subscription only renews when the provider says it did.** Nothing extends a period on a timer — the alternative is giving away membership to somebody whose card was declined.
2. **Cancelling never takes away time already paid for.** It stops the next payment; the entitlement runs to the end of the period, and the interface says the date before you press the button.
3. **Period perks are paid once per period.** Providers deliver the same webhook twice sooner or later, so a renewal that does not move the period forward is treated as the duplicate it is and pays nothing.

A lapse sweep runs once a minute and retires memberships whose period ended, so a missing webhook cannot become a free membership forever. It allows **three days of grace** for a late provider — cutting somebody off the second their period ends turns a provider's slow queue into a support ticket — but a member who actually cancelled is retired immediately.

Entitlements

One table answers "may this account do X right now" — ad-free, premium pass, subscription, VIP. Everything that grants a benefit writes a row there rather than flipping a private flag, so revoking is one operation and support can see where a benefit came from. Granting the same entitlement twice **extends** rather than duplicating.

Advertising

House-served from campaigns an advertiser books. Three deliberate choices:

- **Ads are drawn, not downloaded.** A creative is a headline, a body line, a call to action and two colours. No third-party script loads, so no ad can track a player across the internet, slow the game down, or look out of place.
- **Frequency is enforced on the server.** Daily caps and minimum gaps live in the database, so clearing app data does not reset somebody's allowance. Interstitials are capped hardest (8/day, 3 minutes apart).
- **Targeting is coarse on purpose.** Country, platform, level band. No behavioural profile, no cross-app identity.

Rewarded video pays only against an impression the server itself issued, only once per impression, and only if enough time passed for the clip to have played.

Promo codes

Redemption is rate limited to 6/minute so guessing is slower than asking. Every validity check — active, in window, uses remaining — lives in a single `UPDATE` with the guards in the `WHERE` clause, so two devices redeeming the last use cannot both win. Unknown and expired codes return the same message, so codes cannot be enumerated.

---

7. Data model

70 tables across 11 forward-only migrations. Each migration runs once inside its own transaction and its checksum is recorded — editing an applied migration is a hard error rather than silent drift between environments.

MIGRATION	COVERS
`001_users_auth`	Users, social accounts, refresh tokens, profiles
`002_matches`	Matches, seats, per-shot events, results, ratings
`003_economy`	Items, inventory, fragments, crates, **coin ledger**
`004_progression`	Achievements, missions, friends, notifications, leaderboards
`005_tournaments`	Tournaments, entrants, bracket matches
`006_moderation`	Reports, cheat flags, bans, mutes, audit log, analytics
`007_seasons_progression`	Seasons, login streaks, fair play, follows, spectators, app config, idempotency keys
`008_collections`	Profile cosmetics, collection milestones, emote state
`009_telemetry_events`	Performance samples, client errors, live events, retention
`010_monetization`	Gems + **gem ledger**, store, purchases, refunds, entitlements, subscriptions, pass, promos, advertisers, campaigns, creatives, impressions, frequency caps
`011_bots_gifts`	Bot accounts and their skill bands, coin gifts with the one-per-day index

Idempotency

Anything that pays out uses one of two patterns, both of which make a retry free rather than expensive:

- A **claim row whose primary key is the thing being claimed** (`pass_claims`, `collection_claims`, `ad_rewards`) — the second insert conflicts and pays nothing.
- An `idempotency_keys` **table** with `ON CONFLICT DO NOTHING RETURNING`, so a duplicate is detected without aborting the surrounding transaction.

---

8. API surface

162 HTTP endpoints across 34 route groups, plus a Socket.IO gateway.

Player routes

`/auth` · `/profile` · `/catalog` · `/inventory` · `/crates` · `/collections` · `/daily` · `/preferences` · `/seasons` · `/social` · `/leaderboard` · `/tournaments` · `/reports` · `/notifications` · `/search` · `/events` · `/store` · `/pass` · `/subscriptions` · `/promos` · `/ads` · `/gifts` · `/telemetry` · `/analytics`

Admin routes

`/admin` · `/admin/config` · `/admin/moderation` · `/admin/events` · `/admin/perf` · `/admin/store` · `/admin/pass` · `/admin/promos` · `/admin/ads` · `/admin/monetization` · `/admin/subscriptions`

Socket events

Client → server: `lobby:quickMatch`, `lobby:createRoom`, `lobby:joinRoom`, `lobby:playBot`, `lobby:ready`, `lobby:cancel`, `game:shot`, `room:leave`, `chat:quick`, `presence:heartbeat`, `watch:list`, `watch:join`, `watch:leave`, `watch:react`

Server → client: `session`, `lobby:queued`, `lobby:roomCreated`, `lobby:cancelled`, `lobby:error`, `game:start`, `game:shot`, `game:over`, `game:timeout`, `game:resume`, `game:error`, `room:state`, `player:disconnect`, `player:reconnect`, `wallet:update`, `chat:message`, `chat:blocked`, `watch:start`, `watch:count`, `watch:list`, `watch:left`, `watch:reaction`

---

9. Client applications

Web (Next.js) — 15 routes

Home · Play · Watch · Lobby · Tournaments · Locker · Collections · Store · Pass · Crates · Leaderboard · Profile · Settings · Login

Interface: 6 themes (Classic, Dark, Neon, Royal, Jungle, Galaxy) driven by CSS custom properties, so a theme change is one attribute write rather than a re-render.

Accessibility: reduced motion, high contrast, scalable text (90–140%), ARIA labels and full keyboard control of the board (arrows to position and power, A/D to aim, space to shoot). Preferences are stored **server-side**, so a player who sets up reduced motion on their phone finds it already applied on the web.

Board control — two gestures kept strictly apart:

- Drag **the striker** to slide it along your base line. Releasing never shoots.
- Drag **anywhere else** to aim; the line points from the striker straight at your pointer. Release to shoot.
- Power is its own slider, so aiming at something close no longer forces a weak shot.

Tutorial: five interactive steps played on the real board with the real engine — aim and shoot, move the striker, keeping your turn, the queen, and fouls. It advances when the player actually does the thing, not on a timer.

Mobile (Expo / React Native) — 6 tabs

Home · Play · Locker · Crates · Store · Profile. The same board, the same engine, drawn with SVG through the shared `Painter` interface, driven by `PanResponder` with the same two-gesture model.

Admin (Next.js) — 10 pages

Overview · Reports · Anti-cheat · Players · Matches · Items · Events · Revenue · Ads · Performance

Charts are hand-drawn SVG rather than a charting library — these render a few dozen points, and a dependency that ships its own layout engine and theme would cost more than it earns.

---

10. Infrastructure

Local development

SERVICE	PORT	NOTES
PostgreSQL	5432	Docker or a local install
Redis	**6380**	Moved off 6379 to avoid a clash with an existing container
API	4000	`npm run dev:api`
Web	3000	`npm run dev:web`
Admin	3001	`npm run dev:admin`
Mobile (web preview)	8081	`npm run dev:mobile`

```
npm install
docker compose -f infrastructure/docker/docker-compose.yml up -d
npm run migrate -w apps/api
npm run seed -w apps/api
npm run dev
```

Seeding is idempotent — it upserts the catalogue, creates a pass season only if none is running, and adds house ad campaigns only if they are missing.

Redis

Used for presence, matchmaking queues, rate limiting and socket fan-out. The API runs without it in development via in-process fallbacks, so a missing Redis degrades rather than breaks.

Production shape

- API behind a load balancer with `trust proxy` enabled, so `req.ip` is the real client
- Socket.IO with sticky sessions or the Redis adapter for horizontal scale
- Managed PostgreSQL with automated backups and PITR
- Managed Redis
- Web and admin deployed as Next.js apps or exported static + Node
- Mobile shipped through EAS Build to the App Store and Play Store
- Secrets injected as environment variables — **never in the repository, never in the frontend bundle**

Environment configuration

VARIABLE	PURPOSE
`DATABASE_URL`, `DB_POOL_MAX`	PostgreSQL
`REDIS_URL`, `REDIS_ENABLED`	Redis
`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`	Separate secrets per token type
`FACEBOOK_APP_ID` / `_SECRET`, `GOOGLE_CLIENT_ID`	OAuth — server-side only
`STRIPE_SECRET_KEY`, `APPLE_SHARED_SECRET`, `GOOGLE_PLAY_*`	Payment verification
`SANDBOX_PAYMENTS`	Development only; forced off in production
`ADS_ENABLED`, `STORE_CURRENCY`	Commercial switches
`CORS_ORIGINS`	Explicit allowlist

The server refuses to start in production with the development JWT secrets still in place.

Data retention

`perf_samples`, `client_errors`, `analytics_events` and `match_events` grow without bound. Pruning is **admin-triggered and logged**, with a per-table floor that cannot be pruned past — removal is always a deliberate act, never a surprise at 3am.

---

11. Security

CONCERN	MEASURE
Passwords	bcrypt, cost 12
Sessions	Short-lived JWT access + rotating single-use refresh tokens
Token reuse	A replayed refresh token is rejected and its family revoked
Input	zod validation on every body, query and socket payload
SQL injection	Parameterised queries throughout; the one dynamic table name comes from a fixed allow-list
Rate limiting	Per-route buckets: auth, shots, chat, checkout, promo redemption
Headers	helmet; CORS restricted to an explicit allowlist
OAuth	Secrets stay server-side; clients send provider tokens for verification
Payments	Receipts verified with the provider; price and product read from our own database
Authorisation	Role checks (`player` / `moderator` / `admin`) on every admin route
Audit	Every staff action written to `audit_logs`
Privacy	Telemetry carries no device fingerprint — a coarse three-way device bucket, no model string, no IP

12. Testing and verification

Automated

SUITE	COUNT	COVERS
`engine.test.ts`	48	Physics determinism, rules, fouls, queen, termination
`progression.test.ts`	26	XP curve, titles, divisions, fair play, login cycle
`monetization.test.ts`	20	Catalogue invariants, pass structure, ad limits
`bot.test.ts`	17	Bot legality, difficulty ladder, roster determinism
`emotes.test.ts`	11	Cooldown, per-minute cap, timeout, isolation
Total unit	**122**	all passing
`e2e-full.ts`	**276 checks**	Every route and socket event, against a live stack
`e2e-tournament.ts`	**27 checks**	A four-player cup played through to a champion
`e2e-team.ts`	**16 checks**	A 2v2 team match on the four-seat table
`e2e-subscriptions.ts`	**32 checks**	Buy, renew, cancel, resume, expire
`e2e-bots.ts`	**28 checks**	Bot roster, all three difficulties, gifting
`load.ts`	**9 checks**	Concurrent boards under load
Total end to end	**388 checks**	all passing

The full sweep tracks which routes it touched and prints a coverage count, so "nothing was missed" is something you read off the output rather than something the suite claims about itself. It currently exercises **156 routes** directly, and the specialised suites cover the rest.

Load

Twelve concurrent boards — eight human, four against a *hard* bot, since the bot's shot search is the only genuinely CPU-hungry thing on the server:

Shots sent	980
Median confirmation	**30 ms**
95th percentile	60 ms
99th percentile	85 ms
Slowest	183 ms
Refused	23, all `rate_limited`

The question this run exists to answer is whether a bot thinking on one table slows down somebody's shot on another. It does not: the search yields between batches, and the numbers above hold with four of them searching at once. Every refusal was the shot rate limiter correctly rejecting a client playing faster than a human can — the test emits at 320 ms and the floor is 250 ms.

What the end-to-end run actually does

Twenty-eight sections, in order: authentication, profile and progression, catalogue, inventory, crates, collections, daily rewards, preferences, seasons, social, leaderboards, notifications, search, events, store, pass, promos, ads, gifts, telemetry, tournaments, reporting, the admin console, a real match, bots, spectating, private rooms, solo practice, and a closing ledger audit.

It signs in two guests, matchmakes them, and plays a **complete match over real sockets** driven by the server's own broadcasts, then verifies:

- Entry fees taken, pot correct, winner paid the pot less rake, loser paid nothing
- **Every balance equals the sum of its ledger**
- XP, trophies, crate award, match history, season, missions
- Daily reward claims once and refuses the second attempt
- Shots out of turn and malformed shots refused
- Duplicate reports refused; players cannot reach the admin API
- Preferences persist; omitted fields untouched
- Notifications, global search
- **Store:** an unverifiable receipt is refused, a verified one grants, a replay grants nothing, gems land in the ledger, ad-free removes ads, a one-per-account product cannot be bought twice
- **Ads:** rewarded served, instant completion pays nothing, minimum gap enforced
- **Pass:** premium locked without purchase, unreached tiers refused
- **Promos:** unknown codes refused, a code cannot be redeemed twice, a disabled code stops working
- **Bots:** all three difficulties start, the opponent really is a bot, practice is unstaked, and the bot takes its own turns; the matchmaking fallback seats one and it pays into the pot
- **Gifts:** the sender is debited and the recipient credited with no coins created, only one a day, guests refused, non-friends refused, amounts bounded
- **Tournaments:** create, open, register, refuse a double registration, withdraw, refuse an empty start, cancel
- **Sessions:** signing out invalidates that refresh token; signing out everywhere invalidates them all

Manual verification, web

Walked through the built production app: first-run tutorial fires and renders the board correctly; a Starter Pack purchase moved coins 2,000 → 52,000 and gems 0 → 40 and then correctly locked the one-per-account product; the pass track renders all 50 tiers with the free/premium split; switching to the Neon theme re-skinned the app instantly; the house ad banner served with its "Ad" label.

Both the web and admin apps build cleanly for production (15 and 10 routes).

Manual verification, mobile

Previewed through `react-native-web` at a 375×812 phone viewport. The six tabs render (Home, Play, Locker, Crates, Store, Profile); the store shows every section with ₹ prices; the Play screen offers the three bot difficulties; and tapping one started a real match against **Rami**, correctly labelled `BOT · easy` in the scoreboard.

Bugs found and fixed during this work

1. **A player's first emote was silently swallowed** — `lastSentAt` initialised to `0` made a brand-new player look like they were inside a cooldown.
2. **The pass's legendary crate never landed** — it collided with the tier-50 milestone reward and was silently dropped by `ON CONFLICT DO NOTHING`.
3. **Global search crashed** — read `username` from `profiles`, where it does not exist; it lives on `users`.
4. **Two queries referenced** `users.is_banned`, which does not exist — bans live in their own table with scope and expiry.
5. **Search did not escape** `LIKE` **wildcards.** `%` and `_` are wildcards, so searching for `bot_` matched `botp` and `botq`, and a lone `%` would have matched every account on the service.
6. **A live season could be settled at any moment.** `settleSeason` only checked that the season was not already settled, never that it had *ended* — so one call paid every placement reward and reset every rating partway through a competition. It now refuses unless the end date has passed, with an explicit `force` for an operator genuinely cutting a season short, logged as such.
7. **The bot roster was rebuilt from** `Math.random` **on every boot.** Seeding looks a bot up by username, so each restart produced new names and quietly created another thirty-six accounts — 152 had accumulated before it was caught. The roster is now generated from a fixed seed, and the strays were deactivated rather than deleted so their ledger history survives.
8. **The end-to-end sweep leaked a product into the live store.** It now retires what it creates, and asserts that the retired product leaves the catalogue.
9. **Tournaments dead-ended after the bracket was drawn.** `reportTournamentResult` was written, correct, and *never called* — nothing connected a finished board back to the slot it was played for, and no match was ever launched from a pairing. Both halves are now wired, and a four-player cup plays through to a champion.
10. **A duplicate renewal webhook paid the period perks twice.** Every renewal notice set a fresh `current_start`, which made the once-per-period guard pass again. A renewal that does not move the period forward is now recognised as the duplicate it is.

---

13. Known limitations

Stated plainly rather than left to be discovered:

- **Payment providers are not connected.** The verification code for Stripe, Apple and Google is written and correct, but no credentials are configured, so purchases currently run through the sandbox verifier.

Production refuses to sell until real credentials are supplied.

- **Provider webhooks are not mounted.** The subscription state machine is complete and tested — renew, cancel, resume, expire, with duplicate suppression — but it is driven through `POST /admin/subscriptions/notice` rather than a signed webhook endpoint, because that endpoint needs a real provider's signing secret to verify against.
- **Not tested on physical phones.** The mobile app runs via `react-native-web` at `localhost:8081`; a physical device could not reach the dev server because this machine's Windows Firewall is GPO-managed and local rules cannot be added.
- **Push notifications** are stored and served through the API but not delivered to devices.
- **The bot plays one shot ahead.** It scores the position it leaves behind, but does not search the opponent's reply. That is why "hard" is beatable — which is intentional, but it is a ceiling, not a polish item.

---

14. Deployment

Four things ship: the API, the web app, the operator console and the mobile app. The first three are containers or Node processes; the fourth goes through the app stores. They share one database and one Redis.

14.1 Before anything else

Set these, or the deploy is wrong in ways that are hard to see:

```
NODE_ENV=production                # turns off sandbox payments, forces real secrets
JWT_ACCESS_SECRET=<32+ random bytes>
JWT_REFRESH_SECRET=<a different 32+ random bytes>
DATABASE_URL=postgres://user:pass@host:5432/carrom?sslmode=require
REDIS_URL=rediss://host:6379
CORS_ORIGINS=https://carrom.example.com,https://admin.carrom.example.com
PUBLIC_WEB_URL=https://carrom.example.com
```

The server **refuses to start** in production with the development JWT secrets still in place. `SANDBOX_PAYMENTS` is forced off when `NODE_ENV=production`, so a build with no payment provider configured cannot sell anything — which is the safe failure, not a bug to work around.

14.2 Database

Migrations are forward-only and run automatically on API boot. For a controlled release, run them first and deploy after:

```
DATABASE_URL=... npm run migrate -w apps/api
DATABASE_URL=... npm run seed -w apps/api     # idempotent: content, store, bots
```

Seeding is safe to re-run. It upserts the cosmetic catalogue and store products, creates a pass season only when none is running, and creates the bot roster only when those usernames are missing.

Provision managed Postgres with automated backups and point-in-time recovery, and managed Redis. The API degrades to in-process fallbacks without Redis, so a Redis outage costs presence and cross-node matchmaking rather than the service.

14.3 The API

```
npm ci
npm run build -w apps/api
NODE_ENV=production node apps/api/dist/index.js
```

Or with the provided container:

```
docker build -f infrastructure/docker/api.Dockerfile -t carrom-api .
docker run --env-file .env.production -p 4000:4000 carrom-api
```

Behind a load balancer, `trust proxy` is already on, so `req.ip` is the real client and the rate limiters work per user rather than per balancer.

Scaling out. A live match is owned by the node holding its sockets. Two options, both fine:

- **Sticky sessions** on the balancer, keyed on the socket connection. Simplest, and what the matchmaking code assumes by default — with several nodes and no stickiness it degrades to per-node matching rather than pairing players who could never share a room.
- **The Socket.IO Redis adapter**, if you want fan-out across nodes.

Health check: `GET /health` reports database and Redis reachability.

14.4 The web app

```
npm run build -w apps/web
npm run start -w apps/web        # or deploy the .next output to any Next host
```

Build-time configuration:

```
NEXT_PUBLIC_API_URL=https://api.carrom.example.com
NEXT_PUBLIC_APP_VERSION=$(git rev-parse --short HEAD)
```

`NEXT_PUBLIC_*` values are **baked into the bundle at build time**, so a change needs a rebuild, not a restart. Nothing secret may go in one — they are readable by anybody who opens the page.

Serve over HTTPS. The board is a drag surface and the viewport disables pinch-zoom, so the app is best served at a real domain rather than an IP.

14.5 The operator console

Same as the web app, from `apps/admin`, pointed at the same API and served on a separate hostname. It holds no secret of its own — staff sign in with ordinary accounts, and the API refuses every admin route unless the account carries the `moderator` or `admin` role. Bootstrap the first admin with `ADMIN_BOOTSTRAP_EMAIL`, then promote others from the console.

Put it behind whatever your organisation uses for internal access — a VPN, an identity-aware proxy, or an IP allowlist. The role check is the authorisation boundary; network restriction is defence in depth.

14.6 The mobile app

Built with **Expo**, so releases go through **EAS Build**.

One-time setup

```
npm install -g eas-cli
eas login
eas build:configure          # writes eas.json
```

Point the app at production in `apps/mobile/app.json` under `extra`:

```
{ "extra": { "apiUrl": "https://api.carrom.example.com" } }
```

Android

```
cd apps/mobile
eas build --platform android --profile production      # produces an .aab
eas submit --platform android --latest                 # uploads to Play Console
```

Google Play needs: a service account JSON for `eas submit`, the app signing key (EAS manages it unless you bring your own), and a Play Console listing. For in-app purchases, create each product in the Play Console with an id matching the `sku_google` on the corresponding `store_products` row, then set `GOOGLE_PLAY_PACKAGE` and `GOOGLE_PLAY_ACCESS_TOKEN` on the **API**, not the app.

iOS

```
cd apps/mobile
eas build --platform ios --profile production          # produces an .ipa
eas submit --platform ios --latest                     # uploads to App Store Connect
```

App Store Connect needs: an Apple Developer account, a bundle identifier, and an App Store Connect API key for `eas submit`. For in-app purchases, create each product with an id matching `sku_apple`, and set `APPLE_SHARED_SECRET` on the API.

Over-the-air updates. JavaScript-only changes can ship without a store review:

```
eas update --branch production --message "Fix the aim guide"
```

Anything touching native code — a new native module, an SDK upgrade, a permission — needs a full build and a store review. Currency, cosmetics and rules live in JavaScript, so most content changes are an OTA update.

Web preview. `npm run dev:mobile:web` runs the same app through `react-native-web` on port 8081, which is how it is tested here. Useful for review; not a substitute for a device build.

14.7 Order of operations for a release

1. Run migrations against production.
2. Deploy the API. It is backward compatible with the previous clients, so this is safe to do first.
3. Deploy web and admin.
4. Ship the mobile build, or an OTA update if nothing native changed.

Migrations are additive and forward-only, so step 1 never breaks the running version — which is what makes this order safe rather than merely conventional.

14.8 What to watch after a deploy

SIGNAL	WHERE	WHAT BAD LOOKS LIKE
Coin ledger	`GET /admin/economy/audit`	`ok: false` — a balance disagrees with its ledger
Gem ledger	`GET /admin/monetization/gem-audit`	same, for gems
Crash rate	Admin → Performance	above ~1% of sessions
Frame rate	Admin → Performance	10th percentile below 30 fps
Failed purchases	Admin → Revenue	a spike in one failure reason
Anti-cheat queue	Admin → Anti-cheat	a sudden rise in flags

Both ledger audits should read `ok: true` at all times. They are the fastest way to tell whether a release broke something that matters.

---

15. Command reference

```
# Setup
npm install
docker compose -f infrastructure/docker/docker-compose.yml up -d
npm run migrate -w apps/api
npm run seed -w apps/api

# Development (all four apps)
npm run dev

# Individually
npm run dev:api      # 4000
npm run dev:web      # 3000
npm run dev:admin    # 3001
npm run dev:mobile   # 8081

# Verification
npm run typecheck                      # every workspace
npx vitest run --root apps/api         # 122 unit tests
npx tsx apps/api/test/e2e-full.ts         # 276 checks, needs the API running
npx tsx apps/api/test/e2e-tournament.ts   # a cup played to a champion
npx tsx apps/api/test/e2e-team.ts         # a 2v2 on the four-seat table
npx tsx apps/api/test/e2e-subscriptions.ts
npx tsx apps/api/test/e2e-bots.ts
MATCHES=8 BOTS=4 npx tsx apps/api/test/load.ts
npx next build                         # in apps/web or apps/admin

# Release
eas build --platform android --profile production   # in apps/mobile
eas build --platform ios --profile production
eas update --branch production                      # JavaScript-only changes
```